

Tecnologia em Jogos Digitais • Inteligência Artificial e Ilusão de Inteligência

Inteligência Artificial e Ilusão de Inteligência

Semana 3 — Aprofundamento da FSM: o padrão State em C#

Módulo 1: Como um NPC decide o que fazer?

Apostila: Parte II, Cap. 3 (aprofundamento)

Micro Game: NPC Decision

IFMS • Semana 03

Objetivos da aula

Ao final de hoje você será capaz de:

- explicar o problema de um `switch` de FSM que cresce demais;
- descrever a estrutura do padrão de projeto State (GoF);
- mapear estado, transição, evento e ação para o padrão State;
- refatorar a FSM da Semana 2 para o padrão State em C#;
- reconhecer o que essa refatoração resolve e o que ela não resolve.

Antes de começar: mudança de rota

⚠ ATENÇÃO

Esta turma **não vai estudar Máquinas de Estado Hierárquicas** (Capítulo 4) neste semestre. O Micro Game segue direto de FSM para Árvore de Comportamento, na Semana 4.

Hoje o assunto não é "como organizar hierarquicamente" — é "como organizar o **código**" da FSM que você já tem.

Retomada da Semana 2

Você já tem uma FSM funcional: estados, transições, eventos e ações, implementados por `enum` / `switch` (ou já pelo padrão State, para quem escolheu esse caminho).

O que acontece com esse código quando o número de estados cresce?



Como estruturar o código da FSM para que ela cresça sem virar um emaranhado de condicionais?

O guarda tem cinco estados. Cada um com sua própria lógica de update. E tudo mora no mesmo `switch`.

O problema: um método que cresce demais

Não é a quantidade de estados — é que **toda a lógica de todos os estados convive no mesmo lugar**.

⚠️ ATENÇÃO

Um `switch` com cinco `case`, cada um com dezenas de linhas, é difícil de ler, difícil de testar isoladamente e fácil de quebrar ao editar um caso vizinho.

A resposta: o padrão de projeto State

Cada estado vira uma **classe própria**, que implementa uma interface comum.

- Uma interface `IState` com `Enter`, `Update`, `Exit`
- Uma classe concreta por estado (`PatrulharState`, `AtacarState` ...)
- Um **contexto** — a FSM — que mantém o estado ativo e delega as chamadas

Tecnologia em Jogos Digitais • Inteligência Artificial e Ilusão de Inteligência

Estrutura do padrão State

IFMS • Semana 03

Mapeando a Semana 2 para o padrão State

Conceito (Semana 2)	Elemento no padrão State
Estado	Classe que implementa <code>IState</code>
Ciclo enter/update/exit	Métodos da interface <code>IState</code>
Transição	Chamada explícita de troca de estado no contexto
Evento/guarda	Condição avaliada dentro do <code>Update</code> do estado ativo

Exemplo: do switch à classe

```
// Antes: um case dentro de um switch gigante
case EstadoGuarda.Atacar:
    if (vida < 0.2f) TrocarPara(EstadoGuarda.Fugir);
    AtacarJogador();
    break;
```

```
// Depois: uma classe isolada
public class AtacarState : IState
{
    public void Enter(Guarda g) { }
    public void Update(Guarda g)
    {
        if (g.Vida < 0.2f) g.TrocarPara(new FugirState());
        g.AtacarJogador();
    }
    public void Exit(Guarda g) { }
}
```

O que o padrão State melhora

- **Isolamento:** a lógica de "Atacar" só existe em `AtacarState`
- **Legibilidade:** cada arquivo tem uma responsabilidade só
- **Testabilidade:** dá para testar um estado sem montar a FSM inteira



O que o padrão State não resolve

OBJETIVOS

O número de regras de transição continua o mesmo — só está mais organizado. A explosão de transições, identificada na Semana 2, segue de pé.

Tecnologia em Jogos Digitais • Inteligência Artificial e Ilusão de Inteligência

Implementação guiada

Refatorando a FSM da Semana 2 para o padrão State.

IFMS • Semana 03

Passo a passo

1. Revisar como sua FSM foi implementada (Animator, `switch` ou já padrão State);
2. Definir a interface `IState` (ou equivalente) com Enter/Update/Exit;
3. Criar uma classe concreta para cada estado já existente;
4. Implementar o contexto (a FSM), que troca entre estados;
5. Testar: o comportamento deve ser **idêntico** ao de antes.

O contexto: quem manda na troca

DICA

O estado decide **quando** pedir a troca (dentro do seu `Update`). O contexto (a FSM) é quem de fato **realiza** a troca, guardando a referência ao estado ativo.



Erro comum

Colocar toda a lógica de transição dentro de cada classe de estado, sem um contexto central que controle qual estado está ativo.

OBJETIVOS

Pergunta-chave: "quem guarda a referência ao estado ativo?" Se a resposta não for "o contexto", reveja o desenho.

Discussão técnica: `switch` versus padrão State

Compare a legibilidade e a testabilidade do código antes e depois da refatoração.

⚠ ATENÇÃO

A quantidade de transições escritas mudou? Não. O que ainda falta resolver é a mesma coisa de sempre: reordenar prioridades e reutilizar comportamento continuam trabalhosos — gancho para a Semana 4.

Resumo da semana

- O `switch` de FSM cresce mal quando o número de estados aumenta
- O padrão State isola cada estado em sua própria classe
- Estado, transição, evento e ação mapeiam diretamente para a estrutura do padrão
- O contexto (a FSM) mantém o estado ativo e delega as chamadas
- A explosão de transições **continua sem solução** — não é isso que o padrão State resolve

Preparação para a Semana 4

Tema: Árvores de Comportamento, Blackboard e Engenharia Reversa

- Ler o Capítulo 6 da Apostila (Árvores de Comportamento) e o Capítulo 14 (Engenharia Reversa)
- Trazer a FSM (padrão State) funcional implementada hoje, no Micro Game NPC Decision



DICA

Pergunta que abre a Semana 4: como tornar a decisão do NPC modular e escalável, migrando direto da FSM para uma Árvore de Comportamento?